

Création automatisée de comptes Active Directory en PowerShell

Un script PowerShell qui lit un fichier CSV et crée d'un seul coup les 22 comptes utilisateurs du domaine, avec leurs attributs, leur mot de passe et leur emplacement dans l'annuaire.

RÔLE	ENVIRONNEMENT	VOLUME	STATUT
Écriture du script, exécution, vérification	Windows Server 2022 (VirtualBox)	22 comptes créés	Projet réalisé

01 · OBJECTIF

Ne pas créer 22 comptes à la main

Créer un compte utilisateur dans Active Directory prend une minute : nom, prénom, identifiant, mot de passe, adresse mail, emplacement dans l'annuaire. Multiplié par 22, cela devient une heure de clics répétitifs — et surtout une heure pendant laquelle une faute de frappe peut se glisser sans que personne ne la voie.

L'intérêt du script n'est donc pas seulement le temps gagné. C'est la **régularité** : les 22 comptes sont construits exactement selon la même règle, avec le même format d'identifiant, la même convention d'adresse mail et le même emplacement. Ce qui est produit une fois correctement est produit correctement 22 fois.

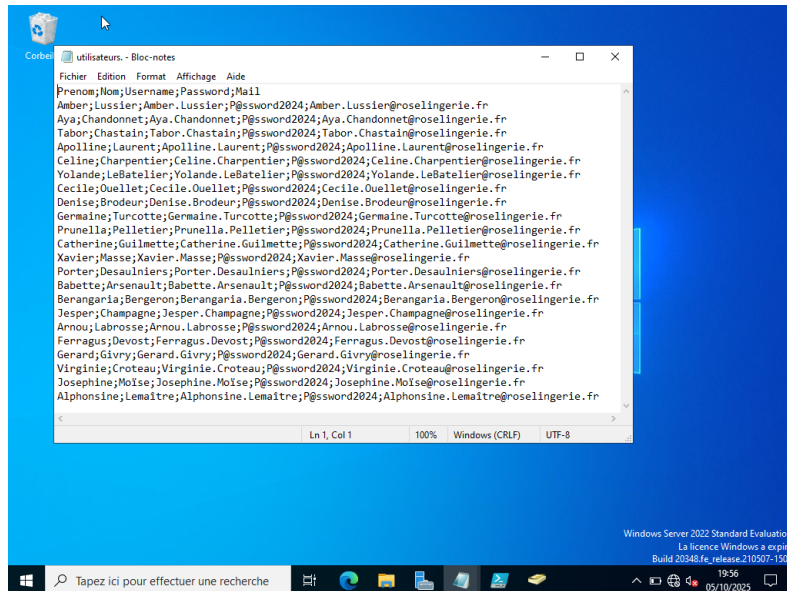
Ce lab s'appuie sur le domaine **RoseLingerie.fr** monté précédemment, et alimente ensuite les stratégies de groupe et les partages, qui s'appliquent à des utilisateurs — donc à des comptes qui doivent d'abord exister.

02 · PRÉPARER LE TERRAIN

Une source de données et une destination

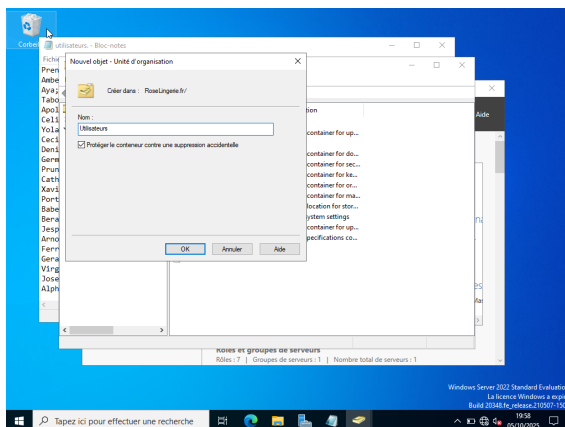
Un script d'automatisation a besoin de deux choses avant d'être lancé : **d'où viennent les données**, et **où vont les objets créés**.

La source est un fichier **CSV** à cinq colonnes — prénom, nom, identifiant, mot de passe et adresse mail — séparées par des points-virgules, une ligne par utilisateur. C'est le format qu'on récupère naturellement d'un export RH ou d'un tableur.

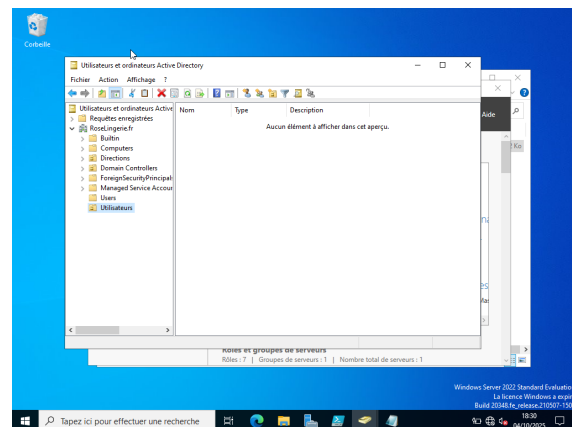


FICHIER SOURCE Le CSV : une ligne d'en-tête, puis 22 lignes d'utilisateurs. L'identifiant suit la convention Prenom.Nom, l'adresse mail la même sur le domaine.

La destination est une **unité d'organisation** dédiée, nommée **Utilisateurs**, créée à la racine du domaine. Regrouper les comptes dans une OU plutôt que de les laisser en vrac n'est pas cosmétique : c'est ce qui permettra plus tard d'appliquer une stratégie de groupe à l'ensemble d'un coup. La case « **Protéger le conteneur contre une suppression accidentelle** » est laissée cochée.



ÉTAPE 1 Création de l'unité d'organisation Utilisateurs à la racine de Roselingerie.fr.



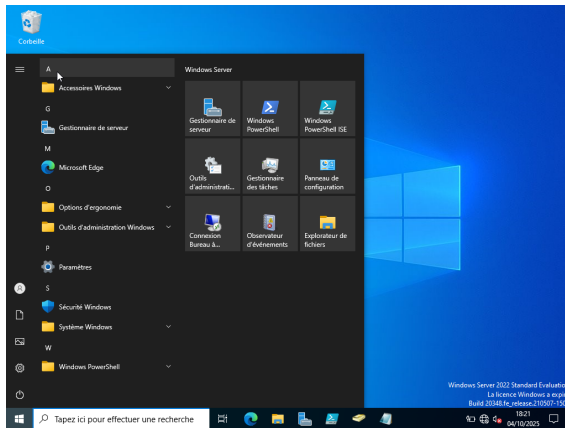
ÉTAPE 2 L'OU existe mais reste vide : c'est l'état de départ, avant exécution.

Cette capture de l'OU vide a son importance : c'est elle qui permettra, à la fin, de prouver que les comptes viennent bien du script et non d'une création manuelle antérieure.

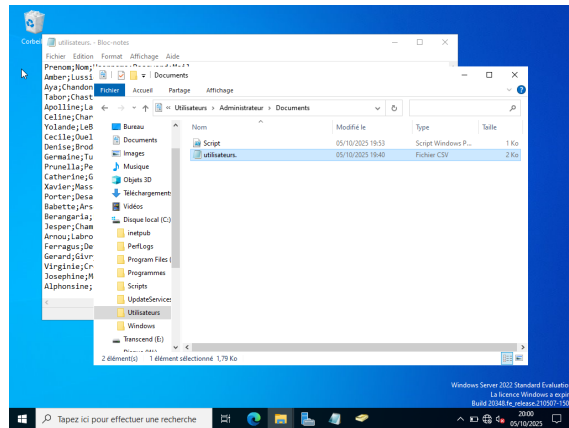
03 · ÉCRIRE LE SCRIPT

Lire, transformer, créer

Le script et le CSV sont placés dans le même dossier, et l'écriture se fait dans **Windows PowerShell ISE**, l'éditeur intégré au serveur : il permet d'écrire et d'exécuter dans la même fenêtre, avec la console de sortie juste en dessous.



ÉTAPE 3 Ouverture de Windows PowerShell ISE depuis le menu Démarrer du serveur.



ÉTAPE 4 Le script et son fichier CSV, placés côte à côte dans le dossier Documents.

La logique tient en trois temps : charger le module Active Directory, lire le CSV, puis boucler sur chaque ligne pour créer un compte.

```
# Importer le module AD
Import-Module ActiveDirectory
$csvPath = "C:\Users\Administrateur\Documents\utilisateurs..csv"
$ouPath = "OU=Utilisateurs,DC=RoseLingerie,DC=fr"

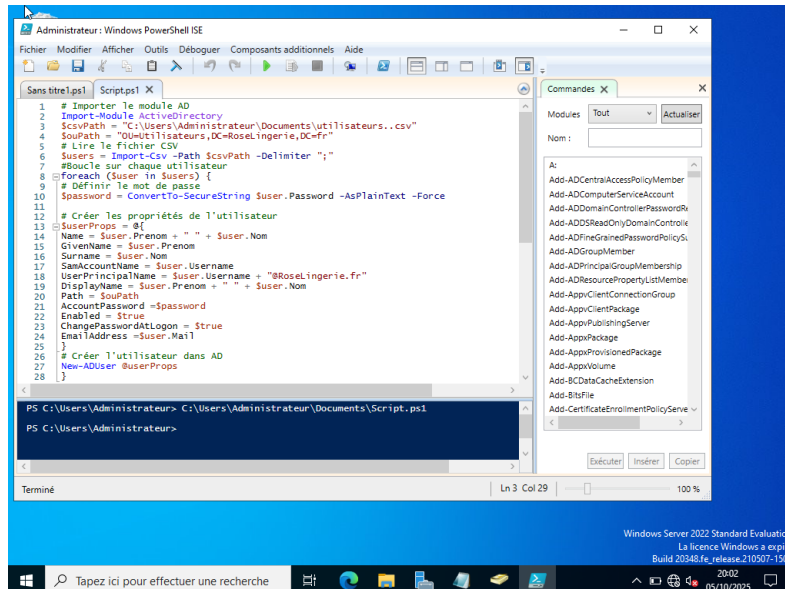
# Lire le fichier CSV
$users = Import-Csv -Path $csvPath -Delimiter ";"

# Boucle sur chaque utilisateur
foreach ($user in $users) {

    # Définir le mot de passe
    $password = ConvertTo-SecureString $user.Password -AsPlainText -Force

    # Créer les propriétés de l'utilisateur
    $userProps = @{
        Name = $user.Prenom + " " + $user.Nom
        GivenName = $user.Prenom
        Surname = $user.Nom
        SamAccountName = $user.Username
        UserPrincipalName = $user.Username + "@RoseLingerie.fr"
        DisplayName = $user.Prenom + " " + $user.Nom
        Path = $ouPath
        AccountPassword = $password
        Enabled = $true
        ChangePasswordAtLogon = $true
        EmailAddress = $user.Mail
    }

    # Créer l'utilisateur dans AD
    New-ADUser @userProps
}
```



EXÉCUTION Le script dans PowerShell ISE. La console ne renvoie aucune erreur et rend la main : les 22 comptes ont été créés en une seule exécution.

04 · LES TROIS LIGNES QUI NE PARDONNENT PAS

Là où ce type de script casse en silence

Un script comme celui-ci paraît simple, mais trois détails suffisent à le faire échouer — ou pire, à le faire réussir à moitié.

Le délimiteur. Import-Csv attend une virgule par défaut. Or un CSV produit dans un environnement français utilise le **point-virgule**. Sans -Delimiter ";", PowerShell lit chaque ligne comme une seule colonne : la boucle s'exécute, ne renvoie pas forcément d'erreur claire, et aucun compte correct n'est créé.

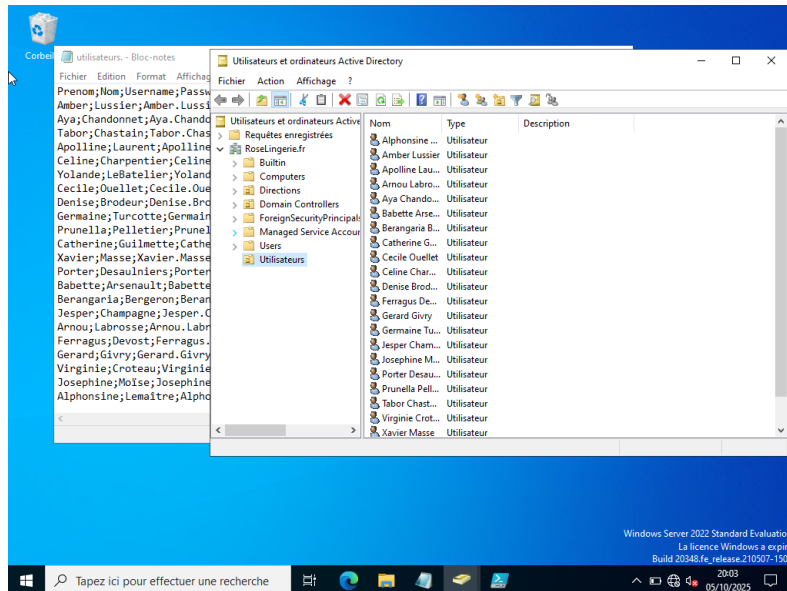
Le mot de passe. New-ADUser refuse une chaîne de caractères simple pour le mot de passe. Il exige un objet sécurisé, d'où la conversion par ConvertTo-SecureString avant chaque création.

Le chemin de destination. \$ouPath doit correspondre exactement à une OU existante, écrite au format LDAP : OU=Utilisateurs,DC=RoseLingerie,DC=fr. C'est pour cette raison que l'unité d'organisation a été créée **avant** l'exécution, et non après. Si elle n'existe pas, chaque appel échoue, un par un, 22 fois.

05 · VÉRIFICATION

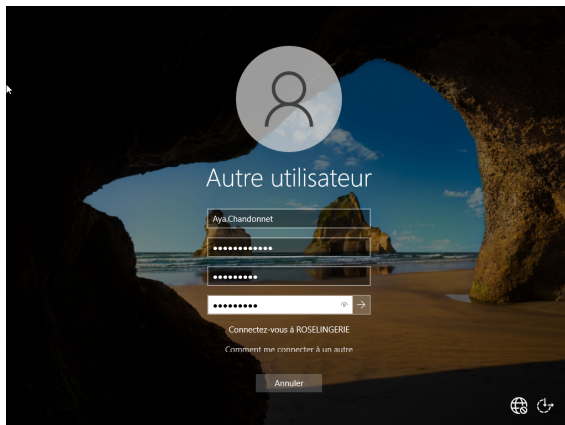
Les comptes existent-ils, et sont-ils utilisables ?

La vérification se fait en trois temps, du plus superficiel au plus concluant. D'abord, les objets sont-ils bien dans l'annuaire ? L'unité d'organisation, vide avant l'exécution, contient désormais l'ensemble des comptes.

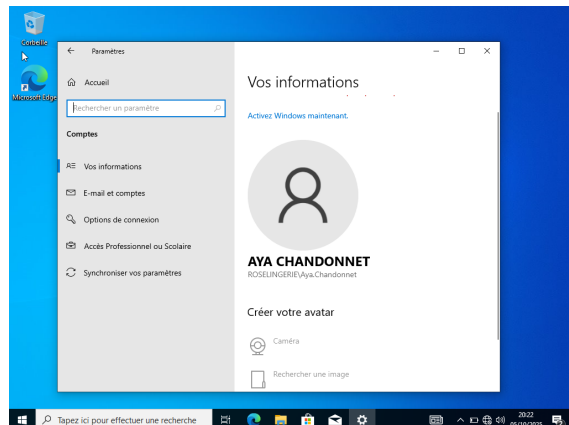


CONTRÔLE 1 L'OU Utilisateurs contient les 22 comptes, chacun repris du CSV avec son prénom et son nom.

Mais un objet présent dans l'annuaire n'est pas encore un compte qui fonctionne. Le vrai test consiste à **ouvrir une session** avec l'un d'eux depuis un poste client. L'écran de connexion apporte au passage une confirmation inattendue : il réclame **trois champs** — l'ancien mot de passe, puis le nouveau et sa confirmation. C'est exactement l'effet de la propriété `ChangePasswordAtLogon = $true` du script, qui impose le changement du mot de passe à la première connexion.



CONTRÔLE 2 Connexion au domaine ROSELINGERIE avec le compte Aya.Chandonnet : changement de mot de passe imposé.



CONTRÔLE 3 Session ouverte. Windows affiche ROSELINGERIEAya.Chandonnet : le compte est bien celui du domaine.

Ce dernier écran est celui qui conclut réellement le projet. Le compte n'a pas seulement été créé : il s'authentifie auprès du contrôleur de domaine, il ouvre une session, et il porte bien le nom du domaine. Le script a produit des comptes utilisables, pas seulement des lignes dans un annuaire.

06 · POINTS CLÉS À RETENIR

Ce qu'il faut anticiper sur ce type de script

POINT CLÉ

Créer la destination avant de lancer le script

Le chemin LDAP indiqué dans le script doit pointer vers une OU qui existe déjà. C'est la dépendance la plus facile à oublier, et celle qui fait échouer les 22 créations d'un coup.

POINT CLÉ

Le CSV français ne se lit pas tout seul

Import-Csv attend une virgule. Un fichier séparé par des points-virgules doit être lu avec -Delimiter ";", sans quoi le script tourne sans rien produire d'exploitable.

POINT CLÉ

Des mots de passe en clair, oui — mais en lab, et pour une seule connexion

Stocker des mots de passe en clair dans un CSV serait inacceptable en production : on générerait un mot de passe aléatoire par compte, sans jamais l'écrire dans un fichier. Ici, c'est un lab isolé, et surtout ChangePasswordAtLogon rend ce mot de passe inutilisable dès la première connexion de l'utilisateur.

POINT CLÉ

Vérifier en ouvrant une session, pas en regardant l'annuaire

Voir 22 lignes apparaître dans la console AD ne prouve que la création des objets. Ouvrir une session avec l'un d'eux prouve que le mot de passe, l'activation du compte et le rattachement au domaine sont corrects.

07 · SUITE ET BILAN

Ce que ce lab m'a apporté

Ces comptes deviennent la matière première des labs suivants : les regrouper pour appliquer des droits sur des dossiers partagés, leur pousser des réglages par stratégie de groupe, ou étendre le script pour affecter chaque utilisateur à un service et à un groupe de sécurité selon une colonne supplémentaire du CSV.

Ce projet m'a surtout fait comprendre ce qu'**automatiser** veut dire concrètement : ce n'est pas écrire du code compliqué, c'est **décrire une règle une seule fois** et la laisser s'appliquer. Le travail se déplace — il ne s'agit plus d'exécuter 22 fois la même tâche, mais de s'assurer que la règle est juste avant de la lancer, et de vérifier son résultat après. C'est aussi ce qui rend l'erreur différente : une faute de frappe manuelle touche un compte, une erreur dans un script les touche tous les 22.

PowerShell	Active Directory	New-ADUser
Import-Csv	Windows Server 2022	Automatisation